

REVISÃO PARA A A1

Aulas 01 a 06

Banco de Dados II — Prof. Alysson

Prova: 10/10/2026 (sábado)

Como usar esta revisão

- Os slides resumem o **roteiro de revisão**, que tem os exemplos completos
- São **10 etapas**: 0 (mapa da prova), 1 a 7 (conteúdo), 8 e 9 (simulado)
- Cada etapa: **resumo** → **exemplo** → **armadilhas** → **perguntas de fixação**
- Tempo total: **cerca de 3 horas** — dá para dividir em vários dias
- Exemplos: uma **pousada hipotética** (**hospede** , **quarto** , **reserva** , **consumo**)
- Dúvida? **Comentário no material do Classroom**, com a etapa e a pergunta

Etapa 0

Mapa da prova

Como é a prova

Item	Como é
Data	10/10/2026, sábado
Formato	escrita, individual , sem consulta , em papel , com caneta
Parte I	10 objetivas (A, B, C, D) $\times$ 0,5 = 5,0
Parte II	1 discursiva = 5,0
Conteúdo	Aulas 01 a 06

Formatos das objetivas: **alternativa única**, **afirmações I a IV** e **asserção-razão**

O que cai e o que não cai

Cai	Não cai
modelo relacional, integridade, PostgreSQL × MySQL	transações e ACID como tema
normalização, restrições, INSERT / UPDATE / DELETE	concorrência e MVCC
junções, subconsultas, conjuntos, consulta com defeito	índices e EXPLAIN
arquitetura do PostgreSQL, bancos distribuídos	backup e segurança
funções, procedimentos, gatilhos (PL/pgSQL)	ferramentas dos laboratórios

PL/pgSQL é o assunto mais pesado da revisão: treine escrever rotinas à mão

Autoavaliação: "eu sei..."

Eu sei explicar...	Senão, vá para
PK × UNIQUE × FK; por que "sempre mais rápido" é falso	Etapa 1
1FN, 2FN, 3FN; se um comando é aceito ou recusado	Etapa 2
LEFT / FULL JOIN , UNION / INTERSECT / EXCEPT , NOT IN com NULL	Etapa 3
WAL, checkpoint, buffers; fragmentação e replicação	Etapa 4
SELECT × CALL ; SUM vazio e COALESCE	Etapa 5
CASE sem ELSE ; laço sem EXIT ; bloco EXCEPTION	Etapa 6
escrever um gatilho; o que NEW vale num DELETE	Etapa 7

Etapa 1

**Modelo relacional, integridade e
PostgreSQL × MySQL**

Termos e chaves

- **SGBD** = o software; **banco de dados** = os dados organizados
- Tabela = **relação** · linha = **tupla** · coluna = **atributo**
- **Domínio** = valores válidos, **incluindo a regra do negócio**

	PK	UNIQUE	FK
Repete?	não	não	pode
Aceita nulo?	não	sim (vários)	se a coluna permitir
Quantas?	uma	várias	várias
Garante	identificar a linha	não repetir	o valor existe do outro lado

PostgreSQL × MySQL sem slogan

- **PostgreSQL:** objeto-relacional, comunidade, licença permissiva (uso comercial OK), versão anual, ~5 anos de suporte
- **MySQL:** Oracle, Community sob GPL, trilhas **LTS** e **Innovation**, MVCC no InnoDB, JSON amplo
- **Beta** é para teste, **não** para produção
- Desempenho exige **carga reproduzível, ambiente controlado e plano** — um teste isolado não prova nada
- Validar na tela **e** declarar no banco: um **completa** o outro

Desconfie de "sempre", "nunca" e "não tem".

Etapa 2

Normalização, restrições e comandos de escrita

As três formas normais

Forma	Exige	Violação típica
1FN	valor atômico, sem colunas repetidas	'português, inglês' ou foto1 , foto2 , foto3
2FN	nada depende de parte da chave composta	chave (cpf, data) e nome depende só de cpf
3FN	nada depende de coluna não chave	numero_quarto → quarto_tipo

- **Chave simples não tem parte** → o problema é de 3FN, não de 2FN
- Correção: o que depende de outra coisa vai para **a tabela dessa coisa**
- "A chave, a chave inteira e nada além da chave"

Restrições: o que cada uma faz

Restrição	Detalhe que cai
NOT NULL	é ela que obriga a preencher
DEFAULT	coluna omitida recebe o padrão
CHECK	com NULL , a linha passa (só reprova o falso)
UNIQUE	aceita vários nulos
FK sem ação	NO ACTION : recusa apagar o pai referenciado
ON DELETE CASCADE	apaga os filhos junto
ON DELETE SET NULL	põe nulo — falha se a coluna for NOT NULL

Comandos de escrita: tudo ou nada

```
INSERT INTO pesquisa_satisfacao (id_reserva, estrelas)
VALUES (2, 4), (3, 9), (4, 5);          -- estrelas: CHECK de 1 a 5
```

```
ERROR:  new row for relation "pesquisa_satisfacao" violates check constraint ...
DETAIL:  Failing row contains (4, 3, 9).
```

- **Nenhuma** das três linhas entrou — o comando é um só
- `UPDATE quarto SET diaria = diaria + 20;` sem `WHERE` → `UPDATE 5` : **todos** os quartos
- Hábito seguro: rode antes o `SELECT` com o mesmo `WHERE`

Etapa 3

**Junções, subconsultas, conjuntos e
consultas com defeito**

Junções e conjuntos

Recurso	Devolve
INNER JOIN	só quem tem par dos dois lados
LEFT JOIN	todas da esquerda (a do FROM)
FULL OUTER JOIN	todas dos dois lados
CROSS JOIN	m × n combinações
UNION / UNION ALL	junta sem / com repetidas
INTERSECT	só o que está nos dois
EXCEPT	o que está no primeiro e não no segundo

Conjuntos: **mesmo número de colunas**, tipos compatíveis

Padrões que resolvem

```
-- quem NUNCA reservou
SELECT h.nome FROM hospede h
  LEFT JOIN reserva r ON r.id_hospede = h.id_hospede
 WHERE r.id_reserva IS NULL; -- Renan Tavares

-- contar mostrando ZERO: COUNT da coluna da direita
SELECT q.numero, COUNT(r.id_reserva) FROM quarto q
  LEFT JOIN reserva r ON r.numero_quarto = q.numero
 GROUP BY q.numero; -- 301 → 0 (COUNT(*) daria 1)

-- acima da média: subconsulta escalar
SELECT numero FROM quarto WHERE diaria > (SELECT AVG(diaria) FROM quarto);
```

Consulta com defeito (1): resultado errado

Sintoma	Causa	Providência
<code>NOT IN</code> devolve 0 linhas	<code>NULL</code> na subconsulta	<code>NOT EXISTS</code>
contagem maior	junção a mais repetiu linhas	tirar a junção ou <code>COUNT(DISTINCT)</code>
total menor	<code>UNION</code> tirou as repetidas	<code>UNION ALL</code>
<code>LEFT JOIN</code> "sumiu" com quem não tem par	filtro da direita no <code>WHERE</code>	filtro no ON

Consulta com defeito (2): erro ou excesso

Sintoma	Causa	Providência
more than one row returned by a subquery used as an expression	subconsulta escalar com várias linhas	filtrar até 1 linha
each UNION query must have the same number of columns	colunas diferentes	igualar as colunas
m × n linhas	CROSS JOIN	INNER JOIN ... ON
quem não tem par não aparece	tabela preservada do lado errado	inverter as tabelas

O filtro no lugar errado

```
SELECT q.numero, r.entrada
FROM quarto q
LEFT JOIN reserva r ON r.numero_quarto = q.numero
WHERE r.entrada >= '2026-08-01';      -- só 2 quartos: virou INNER JOIN
```

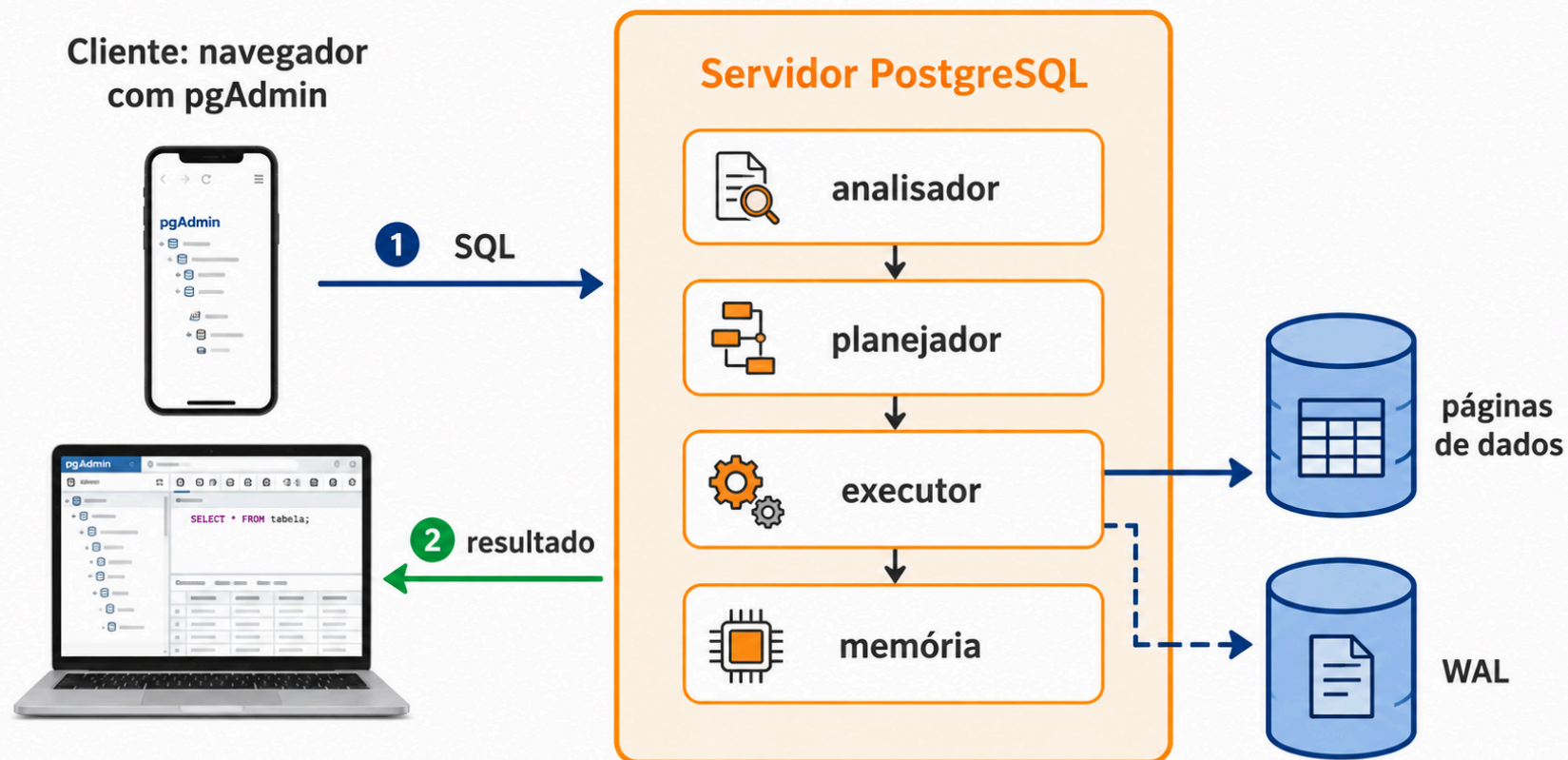
```
SELECT q.numero, r.entrada
FROM quarto q
LEFT JOIN reserva r ON r.numero_quarto = q.numero
                     AND r.entrada >= '2026-08-01';      -- os 5 quartos
```

Sem par, `r.entrada` é `NULL` → a comparação não é verdadeira → o `WHERE` descarta

Etapa 4

Arquitetura do PostgreSQL e bancos distribuídos

O servidor por dentro



Arquitetura (1): processos, memória e nomes

Peça	O que lembrar
cliente × servidor	a tabela fica no servidor , não no cliente
processos	um backend por conexão
buffers compartilhados	páginas em memória: a 2ª execução pode ser mais rápida
esquema	espaço de nomes: <code>recepcao.pagamento</code> e <code>financeiro.pagamento</code> convivem
catálogo	metadados (<code>information_schema.tables</code>), não os seus dados

Arquitetura (2): disco e recuperação

Peça	O que lembrar
páginas	8 KiB (<code>SHOW block_size</code> → 8192)
<code>pg_relation_filepath</code>	só para observar — não edite o arquivo
WAL	registra antes das páginas; refaz o confirmado após a queda
checkpoint	limita o trecho de WAL a refazer na recuperação
backup	o WAL não substitui o backup

Bancos distribuídos

Técnica	Separa	Reconstrói por
fragmentação horizontal	linhas (unidade, região, período)	união
fragmentação vertical	colunas (a chave fica nos dois)	junção pela chave
híbrida	as duas ao mesmo tempo	união + junção
replicação	nada: cópias em vários nós	— (manter coerentes)

- Vários clientes num **único** servidor = cliente-servidor **centralizado**
- **Transparência:** o usuário consulta a tabela lógica sem saber onde está cada linha

Etapa 5

Funções e procedimentos em PL/pgSQL

A anatomia de uma função

```
CREATE OR REPLACE FUNCTION fn_total_consumo(p_reserva_id bigint)
RETURNS numeric          -- função: obrigatório
LANGUAGE plpgsql
AS $$
DECLARE                  -- opcional
    v_total numeric;
BEGIN                    -- NÃO abre transação
    SELECT COALESCE(SUM(valor), 0) INTO v_total
    FROM consumo
    WHERE id_reserva = p_reserva_id;
    RETURN v_total;      -- obrigatório em função que devolve valor
END;
$$;
```

Prefixos: `fn_` · `sp_` · `p_` parâmetro · `v_` variável · `c_` cursor · `fn_tg_` gatilho

Função × procedimento

	Função	Procedimento
Chamada	<code>SELECT fn_x(1);</code> — também no <code>WHERE</code> e no <code>FROM</code>	<code>CALL sp_x(1);</code>
<code>RETURNS</code>	obrigatório	não tem
Devolve	sempre	só por <code>INOUT</code>
<code>COMMIT</code> no corpo	não (<code>invalid transaction termination</code>)	sim
Ligado a gatilho	sim (função gatilho)	não

Chamar do jeito errado dá erro — e a mensagem ensina

```
CALL fn_total_consumo(1);  
ERROR:  fn_total_consumo(integer) is not a procedure  
HINT:   To call a function, use SELECT.
```

```
SELECT sp_lancar_consumo(4, 'lavanderia', 35.00, NULL);  
ERROR:  sp_lancar_consumo(integer, unknown, numeric, unknown) is a procedure  
HINT:   To call a procedure, use CALL.
```

SUM vazio, COALESCE e SELECT INTO

Chamada (reserva sem consumo)	Resultado
<code>SELECT SUM(valor) INTO v_total ...</code>	NULL (coluna vazia)
<code>SELECT COALESCE(SUM(valor), 0) INTO v_total ...</code>	0
<code>COUNT(*)</code> de nenhuma linha	0

- `SELECT ... INTO` **sem linha** → a variável fica **NULL**, **sem erro**
- Na lista de colunas, a função roda **linha a linha**:
`SELECT id_reserva, fn_total_consumo(id_reserva) FROM reserva;`
- Reserva 1: $95,00 + 22,50 = 117.50$ · reservas 4 e 5: **0**

RETURN e valor-padrão

- Função que chega ao `END` **sem RETURN** → **erro** `control reached end of function without RETURN` (não devolve `NULL`)
- `p_percentual numeric DEFAULT 15` : **omitido** vale 15; **informado** vale o informado
 - `fn_diaria_promocional(240.00)` → **204.00** · `fn_diaria_promocional(240.00, 30)` → **168.00**
- Parâmetro com padrão vem **depois** dos sem padrão

Procedimento que devolve valor: **INOUT**

```
CREATE OR REPLACE PROCEDURE sp_lancar_consumo(p_reserva_id bigint, p_descricao text,  
                                              p_valor numeric, INOUT p_consumo_id bigint)  
LANGUAGE plpgsql AS $$  
BEGIN  
    INSERT INTO consumo (id_reserva, descricao, valor) VALUES (p_reserva_id, p_descricao, p_valor)  
    RETURNING id_consumo INTO p_consumo_id;  
END; $$;  
CALL sp_lancar_consumo(4, 'lavanderia', 35.00, NULL); -- devolve p_consumo_id = 5
```

- O **CALL** mostra uma linha com o **valor final** do parâmetro **INOUT**
- **RETURNING ... INTO** guarda o id gerado sem um **SELECT** a mais

RETURNS TABLE e mais pontos de prova

- RETURNS TABLE (...) → chamada **no FROM** ; linhas com **RETURN QUERY**
- Os nomes do RETURNS TABLE viram **variáveis**: sem apelido, column reference "numero_quarto" is ambiguous → escreva r.numero_quarto
- LANGUAGE sql : basta para **uma consulta**; sem variáveis, IF ou EXCEPTION
- Volatilidade: IMMUTABLE (só argumentos) · STABLE (lê tabela) · VOLATILE (padrão)
- CREATE OR REPLACE **não troca o tipo de retorno** → DROP FUNCTION antes
- O código fica no **catálogo** (pg_proc), não num arquivo do cliente
- **Sobrecarga**: mesmo nome, parâmetros diferentes → DROP FUNCTION fn_x(integer, date);

Etapa 6

Controle de fluxo, exceções e cursores

Decidir

Recurso	Armadilha
<code>IF ... ELSIF ... ELSE ... END IF;</code>	é ELSIF ; o primeiro verdadeiro vence
<code>CASE ... END CASE;</code>	sem <code>ELSE</code> e sem <code>WHEN</code> que case → <code>ERROR: case not found</code>

Ordem do `IF` : testando `>= 3` antes de `>= 7` , 10 noites viram `'média'` — **sem erro**

Repetir

Recurso	Armadilha
LOOP ... END LOOP;	precisa de EXIT WHEN
WHILE cond LOOP	testa antes de cada volta
FOR i IN 1..n LOOP	i não é declarada
FOR v IN SELECT ... LOOP	cursor implícito: abre e fecha sozinho
FOUND	o último SELECT INTO / UPDATE / FETCH achou linha?

Falar, interromper e capturar

Comando	Efeito
<code>RAISE NOTICE</code>	aviso; continua
<code>RAISE EXCEPTION ... USING HINT = '...'</code>	interrompe ; nada do comando fica gravado
<code>EXCEPTION WHEN cond THEN</code>	captura: o erro não sobe , o bloco é desfeito, segue pelo tratador
<code>SQLSTATE</code> / <code>SQLERRM</code>	código (<code>23505</code>) e mensagem

```
CALL sp_cadastrar_hospede('Outra Paula', '100000000009'); -- CPF repetido
NOTICE:  O CPF 100000000009 já está cadastrado.
NOTICE:  Código: 23505 – duplicate key value violates unique constraint "hospede_cpf_key"
CALL                                     ← terminou sem erro, e nada foi gravado
```

Cursor explícito: quatro passos

```
DECLARE
    c_quarto CURSOR (p_t text) FOR SELECT numero, diaria FROM quarto WHERE tipo = p_t;
    v_linha record;
BEGIN
    OPEN c_quarto(p_tipo);           -- 1. abre
    LOOP
        FETCH c_quarto INTO v_linha; -- 2. traz a próxima linha
        EXIT WHEN NOT FOUND;         -- 3. SEM ISTO, O LAÇO NUNCA PARA
        RAISE NOTICE 'Quarto %', v_linha.numero;
    END LOOP;
    CLOSE c_quarto;                  -- 4. libera
END;
```

Se um único `UPDATE` ou `SELECT` resolve, **prefira o comando ao cursor**

Etapa 7

Gatilhos

As duas peças

1. A FUNÇÃO GATILHO

```
CREATE FUNCTION fn_tg_exemplo()  
RETURNS trigger  
LANGUAGE plpgsql AS $$  
BEGIN ... RETURN NEW; END;  
$$;
```

sem parâmetros · devolve o tipo **trigger**

2. O CREATE TRIGGER

```
CREATE TRIGGER tg_exemplo  
AFTER UPDATE ON produto  
FOR EACH ROW  
WHEN (condição)  
EXECUTE FUNCTION fn_tg_exemplo();
```

quando · qual evento · qual tabela · por linha

tabela produto

INSERT · UPDATE · DELETE

o servidor dispara sozinho

O esqueleto que cai na discursiva

```
CREATE OR REPLACE FUNCTION fn_tg_nome()      -- parênteses VAZIOS
RETURNS trigger                             -- tipo trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF <regra violada> THEN
        RAISE EXCEPTION 'mensagem clara';
    END IF;
    RETURN NEW;                               -- em DELETE: RETURN OLD
END;
$$;

CREATE TRIGGER tg_nome
BEFORE INSERT OR UPDATE ON tabela
FOR EACH ROW
EXECUTE FUNCTION fn_tg_nome();
```

OLD e NEW: quem é NULL

operação	OLD	NEW
INSERT	NULL não há linha antiga · sem erro	a linha que está entrando
UPDATE	a linha como estava	a linha como vai ficar
DELETE	a linha que está saindo	NULL não há linha nova · sem erro

NULL, e não erro

- Desde o **PostgreSQL 11**: `NEW` num `DELETE` vale **NULL** ; `OLD` num `INSERT` vale **NULL**
- A função **segue em frente** com valores vazios, **sem aviso**
- `record "new" is not assigned yet` era do **PostgreSQL 10 ou anterior**

Log AFTER com <code>OLD.id_reserva</code> e <code>NEW.id_reserva</code>	<code>reserva_old</code>	<code>reserva_new</code>
INSERT	(vazio)	6
UPDATE	6	6
DELETE	6	(vazio)

Para registrar uma exclusão, leia **OLD**

Momento e nível

	BEFORE	AFTER
Roda	antes de gravar	depois de gravar
Alterar NEW	tem efeito	não tem efeito
Vocação	validar e corrigir	registrar e propagar

Nível	Dispara
FOR EACH ROW	uma vez por linha; tem OLD / NEW
FOR EACH STATEMENT	uma vez por comando; é o padrão se omitir

O que devolver

Gatilho	Devolva
BEFORE INSERT / BEFORE UPDATE , por linha	NEW — NULL cancela a linha
BEFORE DELETE , por linha	OLD — NULL cancela a linha
AFTER e todo FOR EACH STATEMENT	ignorado: RETURN NULL

- RETURN NULL num BEFORE → INSERT 0 0 , **sem erro**: a linha some em silêncio
- RETURN NEW num BEFORE DELETE → NEW é NULL → a exclusão é cancelada
- Regra de negócio violada se comunica com **RAISE EXCEPTION**

Exemplo: quarto sem datas sobrepostas

```
CREATE OR REPLACE FUNCTION fn_tg_impedir_conflito()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF EXISTS (SELECT 1 FROM reserva r
               WHERE r.numero_quarto = NEW.numero_quarto
                  AND r.entrada < NEW.saida
                  AND NEW.entrada < r.saida) THEN
        RAISE EXCEPTION 'O quarto % já está reservado entre % e %.',
            NEW.numero_quarto, NEW.entrada, NEW.saida
            USING HINT = 'Escolha outro quarto ou outras datas.';
    END IF;
    RETURN NEW;
END;
$$;
```

Ligando e testando

```
CREATE TRIGGER tg_reserva_conflito
  BEFORE INSERT ON reserva
  FOR EACH ROW
  EXECUTE FUNCTION fn_tg_impedir_conflito();
```

- **Por que gatilho?** O `CHECK` só enxerga a linha nova; a regra precisa ler **as outras**
- **BEFORE** : é validação — a linha errada nunca chega à tabela
- **FOR EACH ROW** : cada reserva é conferida, e só o nível de linha tem `NEW`
- Quarto 101 reservado de 03 a 06/07: entrar em **05/07** → **recusado**; entrar em **06/07** →
`INSERT 0 1`

Quando a regra é violada

```
ERROR:  O quarto 202 já está reservado entre 2026-08-02 e 2026-08-05.  
HINT:  Escolha outro quarto ou outras datas.  
CONTEXT:  PL/pgSQL function fn_tg_impedir_conflito() line 7 at RAISE
```

1. a **função para** no `RAISE EXCEPTION` (o `RETURN NEW` não roda)
2. o **comando inteiro é cancelado** — num `INSERT` de duas linhas, a válida também não fica
3. **nada** daquele comando é gravado (a tabela continuou com 6 reservas)
4. quem executou recebe **ERROR**, **HINT** e **CONTEXT**

Armadilhas de gatilho

- Parâmetro na função gatilho ou `RETURNS boolean` → erro na criação
- `NEW` num `DELETE` ou `OLD` num `INSERT` → `NULL`, sem erro
- `RETURN NULL` no `BEFORE` → linha some sem erro
- Alterar `NEW` num `AFTER` → não grava nada
- Esquecer `FOR EACH ROW` → vira gatilho de comando, `NEW` fica nulo e a regra **deixa de valer**, sem erro
- `RAISE NOTICE` para barrar → o dado **entra**
- Gatilho na tabela errada → ele vai na tabela **que recebe o comando**
- O gatilho **não corrige o passado**; disparos da mesma tabela seguem a **ordem alfabética** do nome

Roteiro da discursiva, à mão

1. Sublinhe a **tabela**, o **evento** e o dado a **consultar**
2. `CREATE OR REPLACE FUNCTION fn_tg_...() RETURNS trigger LANGUAGE plpgsql
AS $$`
3. `DECLARE + SELECT ... INTO`, se precisar de um valor de outra tabela
4. `IF regra violada THEN RAISE EXCEPTION '...'; END IF;`
5. `RETURN NEW; (ou OLD em DELETE) · END; $$;`
6. `CREATE TRIGGER ... BEFORE ... ON tabela FOR EACH ROW EXECUTE FUNCTION
...;`
7. Explique **por que** `BEFORE`, o evento e `FOR EACH ROW`
8. Explique o **efeito da violação** (os 4 pontos)

Etapas 8 e 9

Simulado — não vale nota

Como fazer o simulado

- **Sem consulta**: sem roteiro, sem slides, sem internet
- **Etapa 8** — questões 1 a 10, cerca de **20 minutos**; anote as letras num papel
- **Etapa 9** — questão 11 **à mão, no papel, com caneta**, cerca de **20 minutos**
- Por que à mão? A prova é **manuscrita**: no papel não há editor para lembrar o `$$`, o `END IF;` ou o `FOR EACH ROW`
- Depois, corrija pela **Resolução comentada do simulado**, no fim do roteiro, e dê a sua nota à discursiva item por item

Boa prova!